

Требования к оформлению репозитория с лабораторными работами

1. Общий принцип

Репозиторий — это не просто место, куда нужно загрузить исходный код.

По репозиторию должно быть возможно понять:

1. **какую задачу решал студент;**
2. **что именно он сделал;**
3. **как он это сделал;**
4. **почему был выбран именно такой способ решения;**
5. **какой результат был получен;**
6. **как этот результат можно воспроизвести.**

Главный критерий:

Если открыть репозиторий через несколько месяцев, не задавая автору дополнительных вопросов, должно быть понятно, как работает проект и как был получен результат.

2. Структура репозитория

Для всех лабораторных работ создаётся **один репозиторий**.

Рекомендуемая структура:

```
student-labs/  
|  
├── README.md  
|  
├── lab01/  
|   ├── README.md  
|   ├── src/  
|   ├── results/  
|   ├── data/  
|   ├── config/  
|   └── requirements.txt  
|  
├── lab02/  
|   ├── README.md  
|   ├── src/  
|   ├── results/  
|   └── data/
```

```
|   └─ requirements.txt
|
|   └─ lab03/
|       └─ ...
|
|   └─ .gitignore
```

Допускается изменение структуры, если этого требует конкретная лабораторная работа, но общий принцип должен сохраняться:

код отдельно, результаты отдельно, описание обязательно.

3. Главный README.md

В корне репозитория обязательно должен находиться файл:

README.md

В нём необходимо указать:

Лабораторные работы

ФИО: Иванов Иван Иванович

Группа: ИБ-41

Дисциплина: Название дисциплины

Преподаватель: ...

Выполненные работы

Работа	Тема	Статус
Лабораторная №1	...	Выполнена
Лабораторная №2	...	Выполнена
Лабораторная №3	...	В работе

Желательно сделать названия лабораторных работ ссылками на соответствующие каталоги.

Например:

[Лабораторная работа №1](./lab01/)

4. Каждая лабораторная работа должна иметь собственный README.md

Это **обязательное** требование.

Недостаточно загрузить:

```
lab01.py
```

или:

```
main.ipynb
```

Необходимо объяснить выполненную работу.

В каталоге каждой лабораторной должен находиться отдельный:

```
README.md
```

5. Что должно быть в описании лабораторной

5.1. Цель работы

Кратко сформулируйте, чему посвящена работа.

Например:

```
## Цель работы
```

```
Изучить работу векторного поиска и сравнить результаты обычного поиска  
и поиска с использованием эмбедингов.
```

Не нужно просто копировать методичку.

Напишите своими словами, **что именно вы исследуете или изучаете.**

5.2. Постановка задачи

Перечислите, что необходимо было реализовать.

Например:

Задача

1. Подготовить набор документов.
2. Получить векторные представления документов.
3. Сохранить их в векторной базе данных.
4. Реализовать поиск похожих документов.
5. Сравнить результаты поиска для нескольких запросов.

5.3. Используемые технологии

Необходимо указать основные инструменты.

Например:

Используемые технологии

- Python 3.12
- Qdrant
- sentence-transformers
- Docker
- Jupyter Notebook

Если используется языковая модель, укажите:

- название модели;
- способ запуска;
- облачная она или локальная;
- основные параметры, если они важны для эксперимента.

Например:

Модель: Qwen3-8B
Температура: 0.2
Максимальный контекст: 8192 токенов

6. Самая важная часть — описание решения

Необходимо писать не только:

что было сделано

но ещё:

как это было сделано и почему именно так.

Плохое описание:

Создал базу данных.
Добавил документы.
Сделал поиск.
Получил результат.

Такое описание практически ничего не объясняет.

Хорошее описание:

Для хранения векторов был выбран Qdrant.

Он используется отдельно от основной программы и запускается через Docker, поскольку это позволяет не устанавливать сервер непосредственно в операционную систему и обеспечивает воспроизводимость среды.

Для построения эмбедингов использовалась модель ...

Размер фрагмента документа был выбран равным 500 символам.

При меньшем размере часть контекста терялась, а при значительно большем размере поиск начинал возвращать слишком крупные фрагменты.

7. Объясняйте принятые решения

Если в лабораторной есть выбор, нужно объяснять его.

Например:

- почему выбрали эту модель;
- почему использовали именно эту библиотеку;
- почему выбран такой размер блока текста;
- почему применён этот алгоритм;
- почему выбран такой способ хранения данных;
- почему используется определённый промпт;
- почему установлена конкретная температура модели;
- почему используется локальная или облачная модель.

Нас интересует не только способность получить работающий код.

Важно показать, что вы **понимаете принятые инженерные решения**.

8. Исходный код

Исходный код необходимо хранить отдельно.

Например:

```
src/  
  main.py  
  embeddings.py  
  search.py
```

или:

```
src/  
  experiment.ipynb
```

Если программа состоит из нескольких частей, желательно разделять их по смыслу.

Не нужно помещать весь проект в один файл на несколько тысяч строк.

9. Результаты работы

Кроме исходного кода необходимо сохранить **результат выполнения лабораторной работы**.

Для этого рекомендуется использовать каталог:

```
results/
```

Там могут находиться:

```
results/  
├─ screenshot01.png  
├─ screenshot02.png  
├─ output.json  
├─ metrics.csv  
├─ response.md  
└─ graph.png
```

Что именно сохранять — зависит от лабораторной работы.

Это могут быть:

- скриншоты;
- графики;
- таблицы;
- результаты запросов;
- ответы языковой модели;
- JSON;
- CSV;
- результаты измерений;
- метрики;
- изображения;
- отчёты;
- логи работы программы.

10. Результат должен быть виден прямо в README

Не стоит заставлять проверяющего самостоятельно запускать программу только для того, чтобы понять, работает она или нет.

Основные результаты необходимо показать непосредственно в описании.

Например:

Результат

Запрос:

> Какие методы используются для защиты персональных данных?

Полученный результат:

> Для защиты персональных данных применяются...

Результаты поиска:

Документ	Score	
---	---	
document_03.txt	0.91	
document_07.txt	0.84	
document_01.txt	0.73	

Для визуального результата можно добавить изображение:

```
![Результат работы](./results/result.png)
```

11. Проведение экспериментов

Если лабораторная предполагает исследование, одного запуска программы недостаточно.

Необходимо показать несколько экспериментов.

Например:

Эксперимент 1

Размер chunk: 200

Результат:

...

Эксперимент 2

Размер chunk: 500

Результат:

...

Эксперимент 3

Размер chunk: 1000

Результат:

...

После этого необходимо сделать вывод:

При размере блока 200 поиск часто теряет контекст.

При размере 1000 в результаты попадает слишком много нерелевантной информации.

Для данного набора документов размер 500 показал наиболее стабильный результат.

12. Выводы

Каждая лабораторная должна заканчиваться выводами.

Плохой вариант:

```
В ходе лабораторной работы я изучил Python и выполнил поставленную задачу.
```

Такой вывод не содержит полезной информации.

Хороший вариант:

Вывод

В ходе работы было установлено, что использование векторного поиска позволяет находить документы не только по совпадению слов, но и по смысловой близости.

При этом качество результата сильно зависит от используемой модели эмбедингов и способа разбиения документов.

В эксперименте уменьшение размера блока со 1000 до 500 символов повысило точность поиска, поскольку крупные блоки содержали слишком много нерелевантного текста.

То есть вывод должен отвечать на вопрос:

Что вы поняли в результате выполнения работы?

13. Как запустить проект

Лабораторная должна содержать инструкцию запуска.

Например:

Запуск

Установить зависимости:

```
```bash
```

```
pip install -r requirements.txt
```

Запустить Qdrant:

```
docker compose up -d
```

Запустить программу:

```
python src/main.py
```

Если нужны переменные окружения:

```
```text  
OPENAI_API_KEY  
QDRANT_URL
```

это тоже необходимо указать.

14. Зависимости

Если используются внешние библиотеки, список зависимостей необходимо сохранить.

Для Python:

```
requirements.txt
```

или:

```
pyproject.toml
```

Для JavaScript:

```
package.json
```

Для проектов с Docker:

```
Dockerfile  
docker-compose.yml
```

Цель простая:

другой человек должен иметь возможность скачать репозиторий и запустить ваш проект.

15. Конфигурация

Конфигурационные параметры желательно хранить отдельно от исходного кода.

Например:

```
config/  
config.yaml
```

или:

```
.env.example
```

Пример:

```
OPENAI_API_KEY=  
QDRANT_URL=http://localhost:6333  
MODEL_NAME=
```

16. Пароли и ключи загружать запрещено

В Git нельзя размещать:

- API-ключи;
- пароли;
- токены доступа;
- логины;
- приватные ключи;
- данные банковских карт;
- персональные данные;
- секреты из `.env`.

Файл:

```
.env
```

необходимо добавить в:

```
.gitignore
```

При необходимости создавайте:

```
.env.example
```

без реальных ключей.

17. Данные

Небольшие учебные наборы данных можно хранить в:

```
data/
```

Например:

```
data/  
├── documents/  
├── questions.json  
└── test.csv
```

Если данные занимают сотни мегабайт или гигабайты, загружать их непосредственно в Git не нужно.

В README необходимо указать:

- откуда получены данные;
 - ссылку на источник;
 - как их скачать;
 - куда поместить.
-

18. Что не нужно загружать в Git

Не нужно хранить:

```
venv/  
.venv/  
__pycache__/  
node_modules/  
.idea/  
.vscode/  
.env  
*.log
```

Также обычно не нужно загружать:

- установленные библиотеки;
- виртуальное окружение Python;
- временные файлы;
- кэш;
- автоматически создаваемые файлы;
- большие модели нейронных сетей.

Для этого используется:

```
.gitignore
```

19. История Git

Git необходимо использовать по назначению.

Не нужно выполнять всю лабораторную работу одним коммитом:

```
lab done
```

Рекомендуемый вариант:

```
Initial project structure  
  
Add document loader  
  
Add embedding generation  
  
Add Qdrant integration  
  
Add search experiments  
  
Add experiment results  
  
Update README and conclusions
```

По истории Git должно быть видно развитие проекта.

20. Названия коммитов

Пишите осмысленные названия.

Плохо:

```
update
fix
123
aaa
work
final
final2
final_final
```

Лучше:

```
Add document chunking

Implement vector search

Add reranking experiment

Fix Qdrant connection

Add laboratory report
```

Допускаются названия на русском языке.

Главное — чтобы было понятно, что изменилось.

21. Использование нейросетей

Использовать нейросети при выполнении лабораторных работ можно, если преподавателем не указано обратное.

Но студент должен понимать код и принятые решения.

Фраза:

«Так написал ChatGPT»

не является объяснением.

Если нейросеть предложила решение, необходимо разобраться:

- как оно работает;
- почему оно работает;
- какие существуют альтернативы;
- какие есть ограничения.

Вы должны быть готовы объяснить любую часть кода своего репозитория.

22. Рекомендуемый шаблон README лабораторной

Для каждой лабораторной можно использовать следующий шаблон:

```
# Лабораторная работа №...

## Тема

Название лабораторной работы.

## Цель

Что необходимо изучить или исследовать.

## Задачи

1. ...
2. ...
3. ...

## Используемые технологии

- ...
- ...
- ...

## Структура проекта

```text

src/
results/
data/
```

### Описание решения

#### Этап 1

Что сделали.

Как сделали.

Почему использовали именно такой вариант.

## Этап 2

Что сделали.

Как сделали.

Почему использовали именно такой вариант.

## Запуск

Инструкция по запуску проекта.

## Эксперименты

### Эксперимент 1

Условия:

Результат:

### Эксперимент 2

Условия:

Результат:

## Результаты

Таблицы, изображения, примеры вывода программы.

## Вывод

Что показала лабораторная работа.

Какие решения оказались удачными.

Какие ограничения были обнаружены.

Что можно было бы улучшить.

---

# 23. Что должно быть в репозитории к моменту сдачи

Минимально:

```text

README.md

```
labXX/  
  README.md  
  src/  
  results/
```

В зависимости от работы:

```
data/  
config/  
requirements.txt  
pyproject.toml  
package.json  
Dockerfile  
docker-compose.yml  
.env.example
```

24. Чек-лист перед сдачей

Перед отправкой ссылки на репозиторий проверьте:

- ☐ Репозиторий открывается.
- ☐ Указаны ФИО и группа.
- ☐ Есть отдельная папка лабораторной.
- ☐ Есть `README.md`.
- ☐ Описана цель работы.
- ☐ Описана задача.
- ☐ Указаны используемые технологии.
- ☐ Есть исходный код.
- ☐ Описано, **что было сделано**.
- ☐ Описано, **как это было сделано**.
- ☐ Описано, **почему выбран именно этот способ**.
- ☐ Сохранены результаты работы.
- ☐ Результаты показаны в README.
- ☐ Есть инструкция запуска.
- ☐ Указаны зависимости.
- ☐ Нет API-ключей и паролей.
- ☐ Добавлен `.gitignore`.
- ☐ Есть выводы.
- ☐ По истории Git можно понять ход выполнения работы.

Главное правило

При оформлении лабораторной отвечайте на пять вопросов:

Что? → Как? → Почему? → Что получилось? → Что из этого следует?

То есть:

Что я сделал?

Как я это сделал?

Почему я сделал именно так?

Какой результат получил?

Какой вывод могу сделать?

Если на эти вопросы можно ответить, просто открыв ваш репозиторий, лабораторная оформлена правильно.